

# DIGITAL PATENT - SBRW-GE: SENSORE AMBIENTALE PASSIVO PURO

Core Totalmente Elastico - SBRW-GE v21 [FULL DYNAMIC]

## FASE 1: HARDWARE TELEMETRY ABSTRACTION

*Estrazione e decodifica dei carichi termici dai sensori di sistema e dalla batteria.*

```
class Telemetry_Abstraction:
    def get_telemetria_hardware(self):
        is_screen_on = self.power_manager.isInteractive()
        watt_calc = volts * amps
        return temp_int, batt, is_screen_on, is_charging, watt_calc
```

## FASE 2: DYNAMIC THERMODYNAMIC COMPENSATION

*Compensazione elastica basata sull'assorbimento in tempo reale e stress chimico.*

```
class Thermodynamic_Compensation:
    def calc_target_delta(self, watt_istantanei, batt, is_charging, is_screen_on):
        chimica_mod = 1.15 if (batt < 15 or batt > 85) else 1.0
        if is_charging:
            target_calcolato = (4.0 + (watt_istantanei * 0.6)) * chimica_mod
        elif is_screen_on:
            target_calcolato = (7.0 + (watt_istantanei * 0.8)) * chimica_mod
        else:
            target_calcolato = (watt_istantanei * 1.0) * chimica_mod
        return target_calcolato
```

## FASE 3: ASYMMETRIC INERTIAL FILTER & NEWTON'S LAW

*Applicazione della legge di Newton sul raffreddamento con dissipazione accelerata.*

```
class Newton_Kinetics:
    def apply_inertial_filter(self, target_delta, delta_attuale):
        self.k_riscaldamento = 0.08
        base_k_raffreddamento = 0.02
        boost_dissipazione = max(0.0, (delta_attuale * 0.005))
        k_raffreddamento = min(0.1, base_k_raffreddamento + boost_dissipazione)

        if target_delta > delta_attuale:
            delta_attuale += (target_delta - delta_attuale) * self.k_riscaldamento
        else:
            delta_attuale += (target_delta - delta_attuale) * k_raffreddamento
        return delta_attuale
```

## FASE 4: PURE ENVIRONMENTAL EXTRACTION

*Isolamento logico del calore hardware e generazione della lettura ambientale reale.*

```
class Pure_Environment:
    def extract_ambient_temp(self, temp_int, delta_attuale):
        t_amb_realtime = temp_int - delta_attuale
        return t_amb_realtime
```

**FIRMA LOGICA DI INTEGRITA' STRUTTURALE (SHA-256):**

**38e61ecd2556d60099bf98ffba73c1edb399c259f5ec1b811a0f711c40a0a37b**

LICENZA: GNU AFFERO GENERAL PUBLIC LICENSE (AGPL-3.0)

Il design logico SBRW-GE e' protetto da licenza AGPL-3.0.

Qualsiasi modifica deve essere resa pubblica.

Copyright (c) 2026 - Digital Patent Progetto.